Per-Technique Reporting Template

Agentic Strut-Defect Detection Comparison · DSC 2026

One copy of this template is completed by each contributing team, one per technique. Shaded cells are for you to fill in. Do not change the metric definitions, the units, or the specimen set, since the comparison across five or six techniques is only valid if every entry means the same thing. Return the completed file plus your raw result files and execution traces.

How to use this template

1. Copy this file and rename it Technique_<yourID>_<shortname>.docx, for example Technique_T3_graph_rebuild.docx.
2. Fill every shaded cell. If a metric does not apply to your technique, write N/A and say why in the notes, rather than leaving it blank.
3. Use the exact units printed next to each field. Convert before entering; do not enter a different unit.
4. Run on the shared specimen set and the frozen model and settings named in Section 1. If you deviated, record it in Section 7; an undocumented deviation invalidates the row.
5. Return the completed template, your per-specimen results file, and your raw execution traces to the coordinating author.

How to use this template 1
1. Shared Configuration (do not change) 3
2. Technique Identity 3
2.1 Place your technique on the abstraction axis 3
3. Method: Steps You Executed 4
4. Detection-Quality Results 4
4.1 Recall by defect class 5
5. Cost and Efficiency Results 7
6. Your Row in the Joint Figures 7
6.1 Master comparison table (coordinator fills across techniques) 8
7. Provenance and Deviations 8
8. Submission Checklist 9
Appendix A. Run Outputs (unified DSC input) 10
Appendix B. All Detections 12

1. Shared Configuration (do not change)

These values are fixed for every technique so that differences in the results reflect the technique and not the setup. They are pre-filled by the coordinating author before distribution. If any is blank, request it before running.

Setting	Value
Specimen set (IDs)	To be filled by coordinator
Ground-truth source	aligned JSON graph (nodes + edges)
Nominal defect levels	0%, 0.1%, 0.5%, 1% missing struts
Model + version	_____
Temperature / reasoning effort	_____
Token price sheet (USD)	_____
Repeats per specimen	_____

Table 1. Frozen configuration shared across all techniques.

2. Technique Identity

Technique ID: T2 *(e.g. T3)*

Short name: Pattern Recognition *(e.g. graph rebuild)*

Contributor(s): **your name(s)** *(names for authorship)*

Affiliation / team: Team RES, LLNL DSSI Data Science Challenge 2026

Contributor(s): **additional contributor(s) or delete line** *(names for authorship)*

Affiliation / team: Team RES

2.1 Place your technique on the abstraction axis

The comparison is organized along one axis: the level of abstraction at which your technique compares the CT to the design, from raw pixels to abstract topology. Mark where yours sits, and name the reference it uses.

Question	Your answer
Abstraction level (pixel / image / geometry / topology / none)	pixel (voxel-level pattern voting)
Design reference used (none / STL / JSON graph / rendered slices)	none: self-referential (the scan's own lattice periodicity)
Registration required? (none / slice-plane / full 3D)	none for detection (lattice vectors measured from the CT). Scoring only: affine re-fit of the supplied JSON
One-line description of the core idea	Compares the scan with its own repeating pattern. A gap covering ≥ 85 % of a strut is missing, a shorter one is broken.

Table 2. Self-classification. This is what lets six techniques be read on a single axis in the joint paper.

3. Method: Steps You Executed

List the concrete steps your technique performs, in order, from loading the CT to emitting the flagged defect list. Every technique must ultimately produce the same three artifacts: a flagged defective-strut list scored against ground truth, a segmentation mask (if your technique produces one), and a complete execution trace. Add or remove rows as needed.

Step	Action performed		Artifact / metric it produces
1	Load & Normalize	Load the CT Tiff stack and even out the brightness so both sides of every slice look the same.	Normalized CT Tiff stack
2	Segmentation	Use Otsu thresholding to separate struts from background, then clean up small specks.	Segmented (binary) Tiff stack
3	Lattice Detection	Measure how the lattice repeats in 3D directly from the scan. No design file is used.	Lattice spacing and orientation
4	Neighbour Voting	Compare each point with the same point in the surrounding repeated cells to predict where material should be.	Expected-material volume
5	Gap Detection	Find places where the neighbours expect material but the scan has none, and group them into 3D gaps.	Candidate gaps
6	False-Alarm Filter	Use a machine-learning classifier to remove candidate gaps caused by noise, bent struts or scan artifacts.	Confirmed gaps
7	Strut Classification	Match each confirmed gap to its strut and measure how much is gone: 85% or more is missing, less is broken.	Flagged defective-strut list (missing / broken)
8	Output	Report the number of missing and broken struts, a 3D view of their locations, and marked slices showing each one.	Defect counts, 3D view, marked slices, defect map (JSON), execution trace

Table 3. Your technique's workflow. Mark which steps produce a metric input (mask, flagged list, trace).

Defect decision rule

State, in one or two sentences, the exact rule by which your technique decides a strut is defective, and the threshold values used. This is required so a reader can reproduce your classification.

A gap is a connected region where at least half of the valid lattice-equivalent neighbours (and at least 2) have material but the scan has none within 2 voxels. It must be ≥ 40 voxels and be accepted by the gradient-boosted false-alarm filter ($p \geq 0.5$). A strut is defective when accepted gaps cover ≥ 2 of 25 samples along its free length (node surfaces excluded). It is MISSING if the covered fraction is ≥ 0.85 and BROKEN otherwise.

4. Detection-Quality Results

Enter your scores against the shared ground truth, averaged over all specimens and repeats. The definitions are fixed and reproduced here so no one computes them differently. TP is a design strut correctly flagged defective; FP a sound strut wrongly flagged; FN a genuine defect missed.

$$Precision = TP/(TP+FP) \quad Recall = TP/(TP+FN) \quad F1 = 2{\cdot}P{\cdot}R/(P+R) \qquad (1–3)$$

$$IoU = |P \cap G| \,/\, |P \cup G| \qquad Dice = 2|P \cap G| \,/\, (|P| + |G|) \qquad (4–5)$$

For techniques that build a graph, also report Edge F1, the F1 of reconstructed edges against design struts after matching nodes by position. *Leave it N/A otherwise.*

> **How to read these scores.** A defect detector can fail in only two ways, and each metric isolates one. *Precision* asks how many of your flagged struts were truly defective, so it falls when you raise false alarms (over-flagging). *Recall* asks how many of the real defects you caught, so it falls when you miss defects (under-flagging).
>
> These two pull against each other: flag only your most certain strut and precision is perfect but recall collapses; flag every strut and recall is perfect but precision collapses. *F1* is the harmonic mean of the two, F1 = 2·P·R/(P+R). The harmonic mean is used instead of a plain average because it is dominated by the smaller value, so a technique cannot score well by winning one at the expense of the other. For example P = 0.90 with R = 0.10 averages to 0.50 but gives F1 = 0.18, correctly marking it as poor. F1 climbs toward 1 only when precision and recall are both high, which is why it is the headline accuracy score.
>
> *IoU and Dice* measure something different: not which struts you flagged, but which voxels you labelled as material, comparing your material set P against the ground-truth set G. They judge segmentation fidelity, whereas precision, recall, and F1 judge the strut-level defect decision. *Edge F1* repeats the F1 idea over graph edges, catching connectivity errors (a hairline gap that splits a strut) that barely change voxel overlap but break the topology.

Metric	Value (mean)	± std	Notes
Precision	0.944	0.035	TP 152, FP 9, FN 42. Std = spread over 5 z-blocks (one specimen)
Recall	0.784	0.062	Reference from the unified input (CT + ct_registered JSON): 90 missing + 104 broken; y-max build-plate face excluded
F1 (headline)	0.856	0.039	z-block mean 0.866. All 18,468 struts incl. the absent y-max face: F1 0.230 (recall 0.131)
IoU	0.333	N/A	Proxy: segmentation vs voxelised as-built design (registered JSON, r = 175 µm). No voxel ground truth exists
Dice	0.499	N/A	Same proxy as IoU
Edge F1 (graph only)	N/A	N/A	Not a graph-building technique

Table 4. Detection-quality results for your technique.

4.1 Recall by defect class

Enter recall separately for each defect class. This table feeds the per-class heatmap that reveals each technique's blind spots, and it is often the most scientifically interesting output, so please complete it fully.

Defect class	Recall	# in ground truth	# your technique found
Missing	0.956	90	86 (87 flagged missing)
Broken / disconnected	0.615	104	64 (74 flagged broken)
Thin / undersized	N/A	N/A	not targeted (fair-comparison scope)
Bent / wavy	N/A	N/A	not targeted; bent struts are suppressed as false alarms
Dross	N/A	N/A	not targeted

Table 5. Per-class recall. If your technique cannot target a class, enter N/A.

5. Cost and Efficiency Results

Record these from your execution trace, not by estimate, and per specimen (mean across the specimen set). The units are fixed; convert before entering. Cost efficiency is derived, but enter your own computed value as a check.

Metric	Unit	Value (mean)	± std
Token usage	thousands (k)	21,844	N/A
Runtime	seconds	134.9	1.4
Tool calls	count	88	N/A
Normalized cost	USD	6.28	N/A
Cost efficiency (F1 / USD)	ratio	0.136	N/A

Table 6. Cost results for your technique, from the execution trace.

What counts as a tool call and a token

- A tool call is one discrete invocation of a tool (segment, visualize, skeletonize, render, etc.). A retry counts as a separate call. Report the total to task completion for one specimen.
- Token usage is input plus output tokens summed over every model call for one specimen. Include reasoning tokens if your model and trace expose them; note in Section 7 whether they are included.
- Runtime is wall-clock from task start to final report for one specimen, on the hardware named in Section 7.

6. Your Row in the Joint Figures

When your Tables 4 to 6 are complete, your technique becomes one marker in the shared frontier plot and one row in the master table. The blank frontier below shows where the six techniques will land; the coordinating author will place your marker from your reported cost and F1, sized by your token usage.

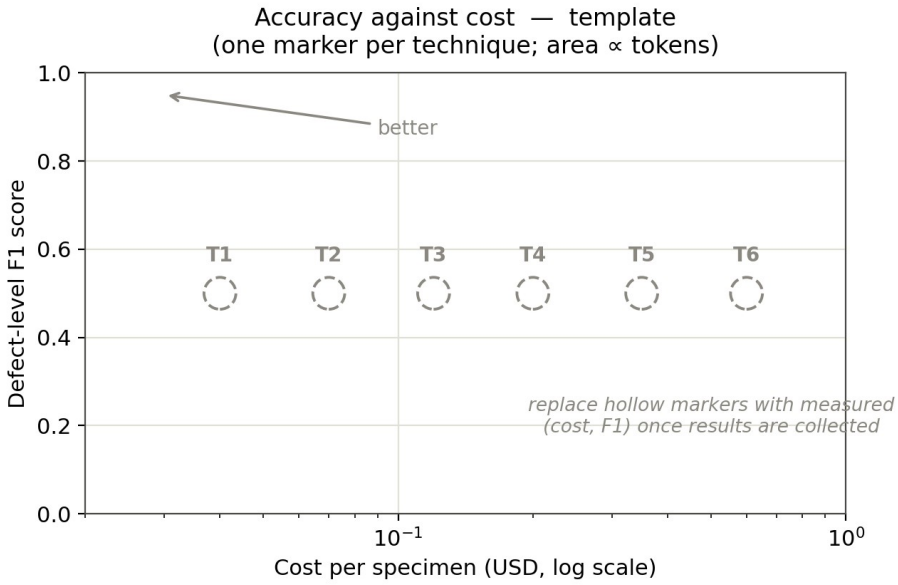


Figure 1. Blank accuracy-against-cost frontier. Hollow markers T1–T6 are replaced by measured points as templates are returned.

6.1 Master comparison table (coordinator fills across techniques)

For reference, this is the consolidated table your row feeds into. You fill only your own column; the coordinating author assembles the rest.

Metric	T1	T2	T3	T4	T5	T6
F1		0.856				
Precision		0.944				
Recall		0.784				
IoU		0.333				
Edge F1		N/A				
Tokens (k)		21,844				
Runtime (s)		134.9				
Tool calls		88				
Cost (USD)		6.28				
F1 / USD		0.136				

Table 7. Master results table across up to six techniques. Placeholder columns; one filled per returned template.

7. Provenance and Deviations

This section protects the fairness of the comparison. Record anything that could make your numbers not directly comparable to another technique's, so it can be accounted for rather than discovered later.

Hardware used: Cloud Linux container, 2 vCPU, 8 GB RAM, no GPU; 3 repeats with identical output (runtime = pipeline wall-clock). Tokens, tool calls and cost are measured from this session's transcript for the algorithm run (detection, evaluation, slice review: 23 model calls, 43 tool calls, 8,053k tokens, $2.52) and the report update (30 model calls, 45 tool calls, 13,790k tokens, $3.76), priced at Claude Opus 5.5 rates ($4/M input, $8/M 1-h cache write, $0.20/M cache read, $20/M output). Over 99 % of the tokens are cache re-reads of a ~410k-token context carried over from earlier work in the same session, so a fresh session would use far fewer. *(CPU/GPU, for runtime context)*

Reasoning tokens included in token count? Yes (billed output tokens include reasoning) *(yes / no)*

Any deviation from the frozen model/settings? Unified DSC input (tif, ct_registered JSON, 0.stl) for specimen 0.5 % dash-1; one specimen, so ± std is the spread over five 152-slice z-blocks. The detector reads only the tif. Reference (evaluation only): the print is a 0 % defect design, so every strut the CT shows ≥ 85 % absent is missing and every strut with no connected material between its nodes is broken. The y-max build-plate face is absent in the specimen; its 1,008 struts (324 missing, 642 broken, 42 sound) are excluded from the headline. With them included: P 0.944, R 0.131, F1 0.230. The GBM false-alarm filter was trained on this specimen (leave-z-block-out) with labels from an earlier reference that also used the 0.5 % STL. Model: Claude Opus 5.5 (claude-opus-5-5), default settings. **Confirm against coordinator's frozen config.** *(describe or 'none')*

Registration method (if any): Detector: none (self-referential). Scoring only: the unified ct_registered JSON, with nodes snapped to the CT node blobs (median shift 1.0 vox, max 5)

Known limitations of your technique: Misses 42 defective struts (39 broken), 32 of them on the lattice's outer faces where few neighbours vote; does not flag the absent build-plate face; hides hairline breaks under ~2 voxels. 9 false alarms, all flagged broken. The noisy top/bottom slices are unreliable. The false-alarm filter is trained on a single specimen

Link to code / commit hash: github.com/reddykingit/llnl_data_science_challenge_2026_TeamRES, branch pattern_recognition @ **commit hash**

Link to raw traces: results/0point5dash1/trace.jsonl (pipeline), review.md, score.json; **Claude Code session transcript**

8. Submission Checklist

- File renamed Technique_<ID>_<shortname>.docx.
- Every shaded cell filled, or marked N/A with a reason.
- All units match those printed in the tables.
- Ran on the shared specimen set and frozen model/settings, or deviations recorded in Section 7.
- Per-specimen results file attached.
- Execution traces attached.
- Code link and commit hash provided.

Citation style note. *For the joint manuscript, in-text citations will be inserted in Mendeley format and the bibliography generated from the shared Mendeley library, so contributors do not need to format references in this template.*

Appendix A. Run Outputs (unified DSC input)

Result: 87 missing struts and 74 broken struts (161 defective struts) in specimen 210127 0.5 % dash-1.

Input. The detector reads only the CT scan, data/210127_Brian_Tran_strut_lattices_0point5dash1 1 Slices.tif. The evaluation reference uses the same scan and data/210127_Brian_Tran_strut_lattices_0point5dash1 1 Slices.ct_registered.json, with the print treated as a 0 % defect design (0.stl): every strut the CT shows ≥ 85 % absent is missing, and every strut with no connected material between its two nodes is broken.

Run. python -m src.strut_defects.pipeline run --input "data/210127_Brian_Tran_strut_lattices_0point5dash1 1 Slices.tif" --out results/0point5dash1 (or the strut-defect-run skill in Claude Code). 134.9 ± 1.4 s over 3 repeats, identical output each time.

Output files (results/0point5dash1/): summary.json (counts); defects.csv and defects_map.json (one entry per defect, linked to its marked slice); defects_3d.png and defects_3d.html (3-D view); marked_slices/ (114 PNG slices with a 6 × 6 grid); marked_slices_viewer_rgb.npy (slideshow); trace.jsonl (execution trace); review.md (visual review); score.json (evaluation).

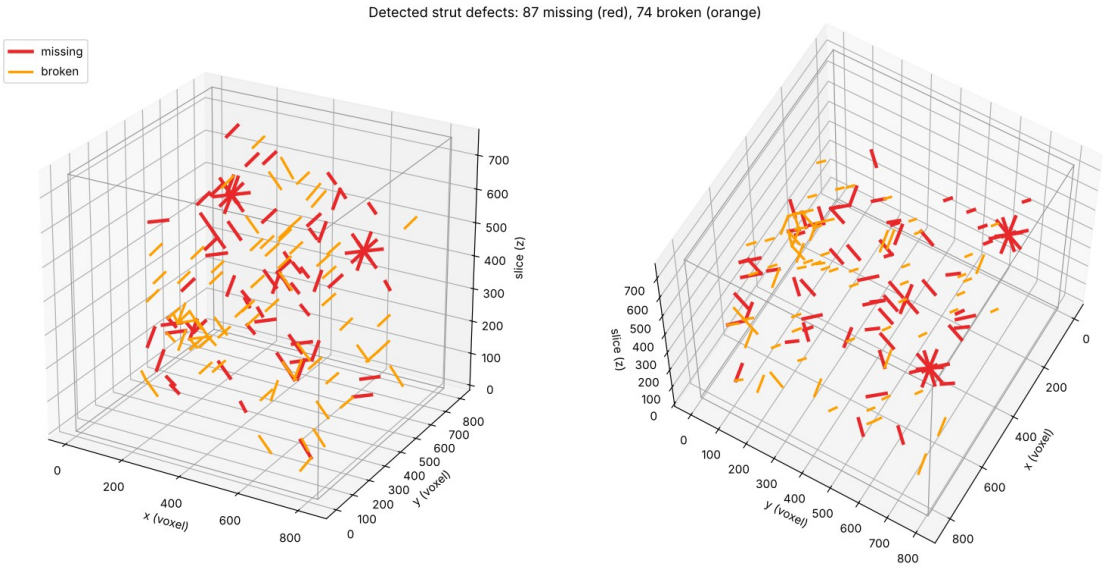


Figure A1. 3-D view of the detected defects: 87 missing (red) and 74 broken (orange) struts, drawn as strut segments in CT voxel coordinates (z = slice). Interactive version: defects_3d.html.

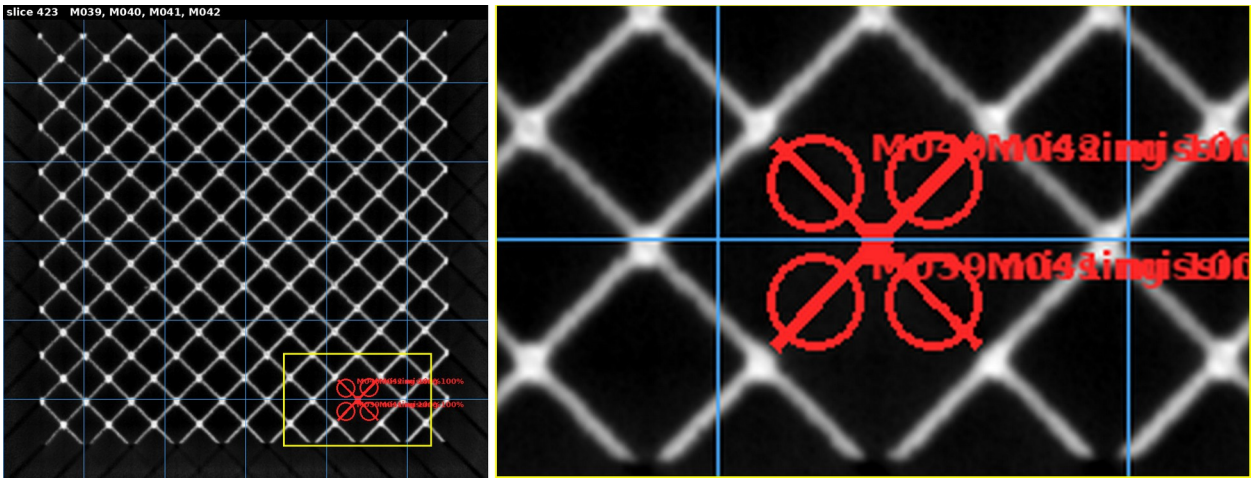


Figure A2. Marked slice 423 with the 6 × 6 grid. Four missing struts (M039–M042, 100 % absent) meet at one node; the right panel is a 2× zoom of the yellow box, where the four labels overlap.

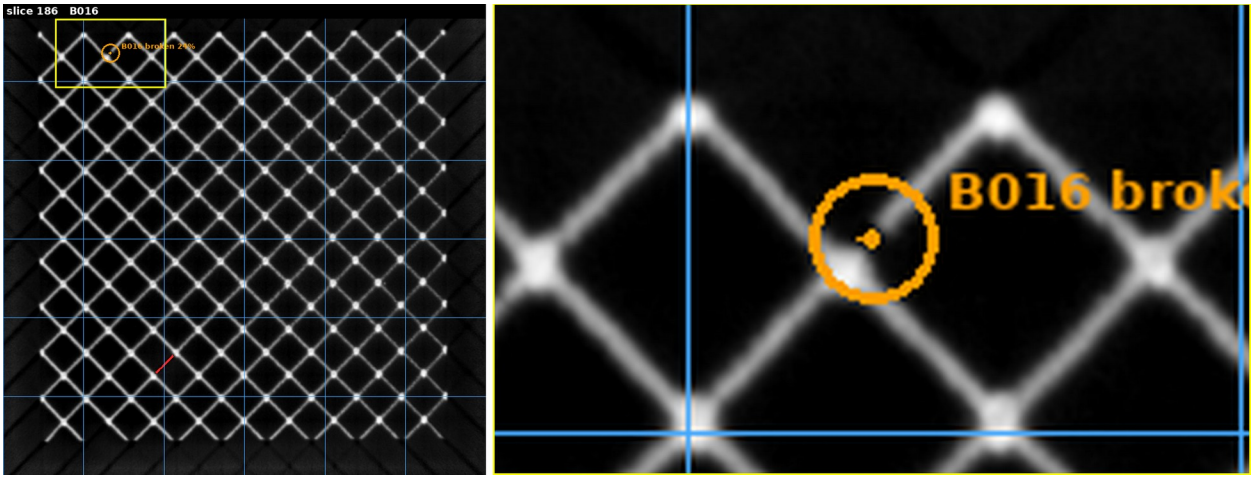


Figure A3. Marked slice 186: broken strut B016 (24 % absent, orange; zoom on the right). The red segment lower left is missing strut M010, marked on slice 187.

Appendix B. All Detections

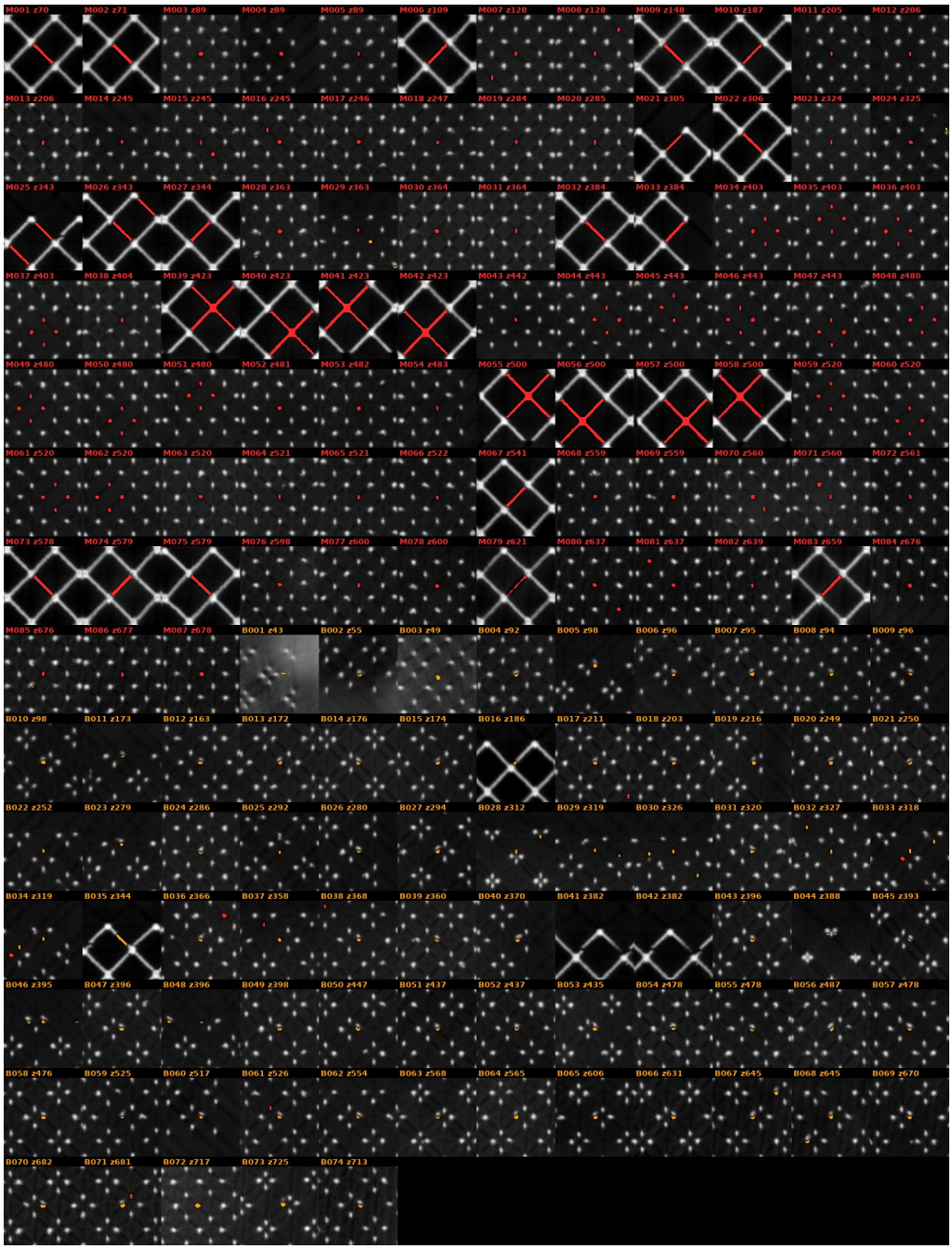


Figure B1. One crop per detection (161), centred on the mark: red = missing, orange = broken; each tag gives the defect id and its marked slice. Full slices: results/0point5dash1/marked_slices/.